

Overall Lab Schedule

Experiments	Content	Week
1	Aircraft War System Analysis and Singleton Pattern	Week 11 Monday: 18:45-22:05 PM Room T2510 
 2	Factory Pattern	Week 12 Thursday: 14:00-17:20 PM Room T2510 
3	Strategy Pattern	Week 14 Tuesday: 14:00-17:20 PM Room T2510 
4	Audio Control and Multithreading	Week 16 Monday: 14:00-17:20 PM Room T2510 



Experiment 2: Factory Pattern

Objective

- Understand the motivation and design intent of the Factory Pattern
- Master the structure of the Factory Pattern and be able to draw its UML class diagram
- Refactor existing code and implement the Factory Pattern in practice



Experiment Tasks

1. Iterative Development Goal

Add a **Boss Enemy Aircraft to the game**

- Including its spawn conditions, movement behavior, firing trajectory, and power-up drop logic upon destruction.

2. Design Pattern Application

Identify scenarios suitable for the **Factory Pattern and implement it**

- Draw the Factory pattern UML class diagram for the aircraft war scenario
- Refactor the code to apply the pattern to the creation of both enemy aircraft and power-up supplies

Experiment Steps: Add Boss Enemy Aircraft

➤ Tasks to be completed:

Boss Enemy Aircraft	
Appearance	Appears when score reaches threshold, multiple times
Movement	hovers and moves left-right above screen
Firepower	Circle fire trajectory, 20 bullets at once
Destroyed	Randomly drops ≤ 2 power-up supplies



Experiment Steps: Factory Pattern (1)

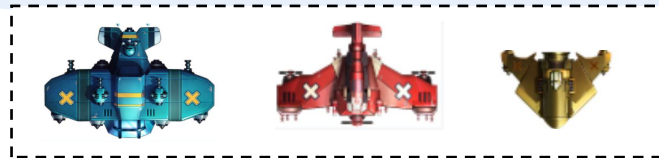
➤ Why do we need the factory pattern?

Scenario Analysis

Requirement:

In the aircraft war game, there are **3 types of enemy aircraft**: mob, elite, and boss.

And **2 types of power-ups**: firepower supply and health supply.



Experiment Steps: Factory Pattern (2)



Let's think about following questions

(1) In which class is the enemy aircrafts currently created? Does this comply with the **Single Responsibility Principle**?

```
if (enemyAircrafts.size() < enemyMaxNumber) {  
    enemyAircrafts.add(new MobEnemy(  
        (int) (Math.random() * (Main.WINDOW_WIDTH - ImageManager.MOB_ENEMY_IMAGE.getWidth())),  
        (int) (Math.random() * Main.WINDOW_HEIGHT * 0.05),  
        speedX: 0,  
        speedY: 10,  
        hp: 30  
    ));  
}
```

Game Class

(2) If a new attribute "defense" is added to enemy aircraft, which parts of the code need to be modified? Does this comply with the **Open-Closed Principle**?

Experiment Steps: Factory Pattern (3)



Let's think about following questions

(3) If other new types of enemy aircraft are added, which parts of the code need to be modified? Does this comply with the Dependency Inversion Principle?



High-level modules should not depend on low-level modules Both should depend on abstractions



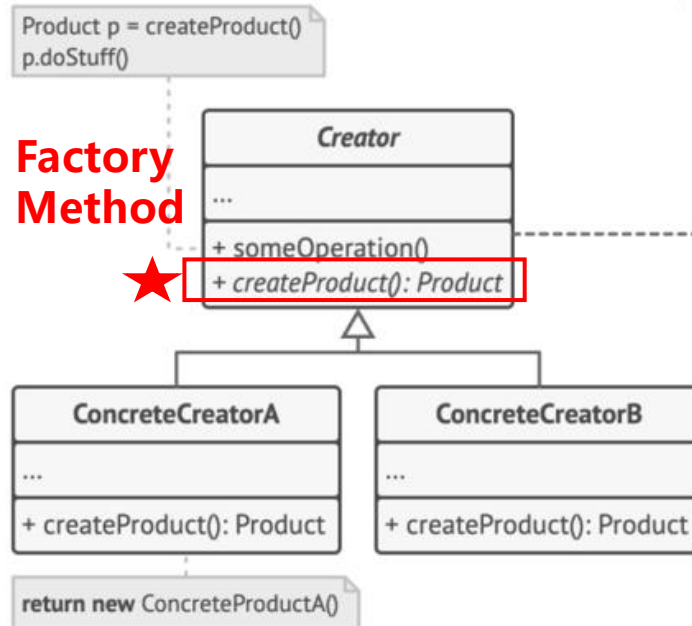
Program to an interface, not an implementation!

Experiment Steps: Factory Pattern (4)

Factory Pattern provides a method in a parent class for creating objects, allowing subclasses to determine the specific type of object to instantiate.

3 Creator
declares the **factory method** that returns a product object. It is an abstract method, forcing each subclass to implement it in a different way.

4 Concrete creators
will override the base factory method to return different types of products.



1 Product
abstraction layer for all products can be defined as an interface or an abstract class

2 Concrete Products
different implementations of the product interface

Factory Line Factory Pattern structure diagram

Product Line

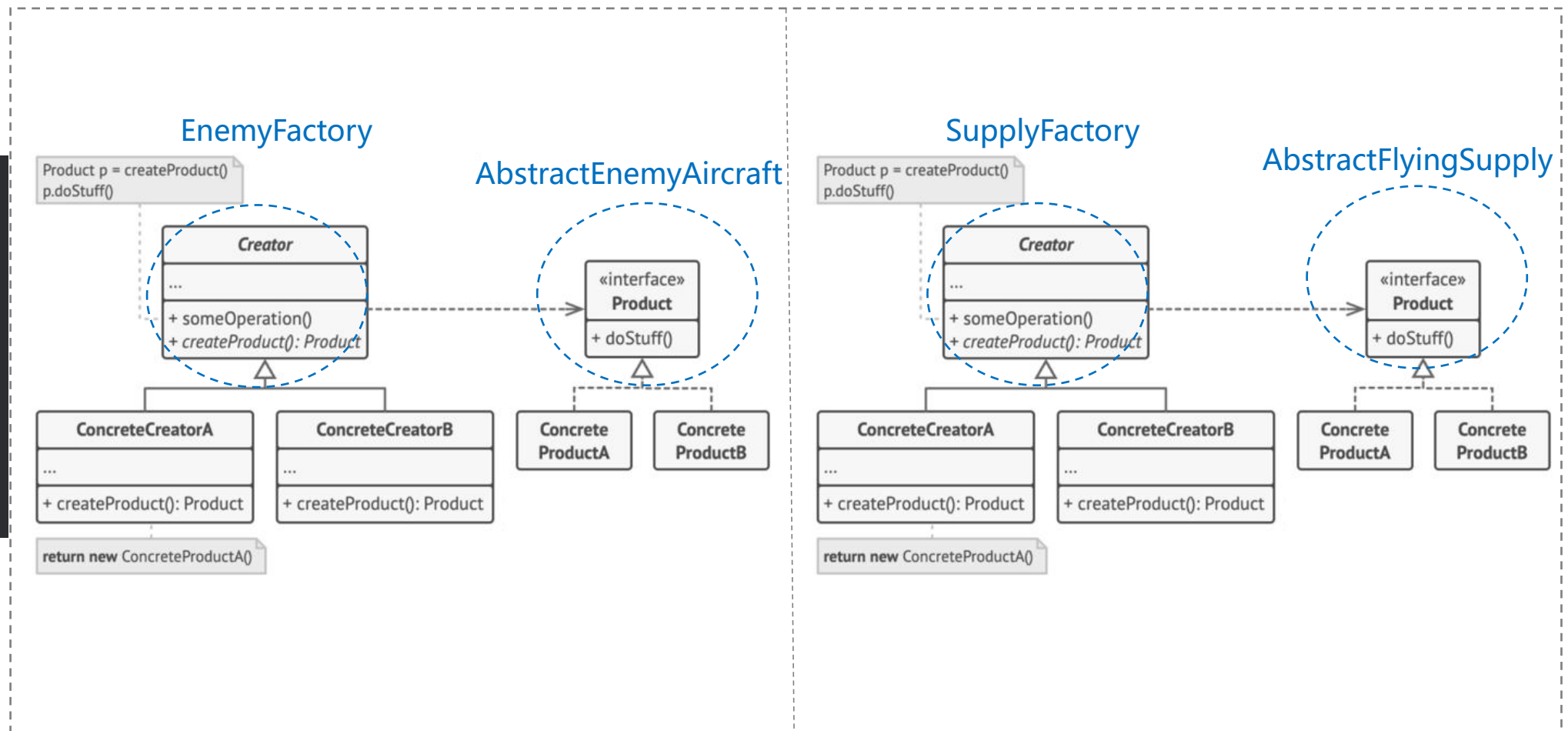
Experiment Steps: Factory Pattern (5)

➤ Tasks to be completed:

- **Draw** the Factory pattern **UML class diagram** for the **enemy aircrafts**. The client side does not need to be drawn.
- **Refactor** code and **create** all **enemy aircraft** using the **Factory Pattern**, ensuring all enemy instances are created through factories.
- Similarly, use PlantUML to **draw UML class diagram** applying the Factory Pattern to **power-up supplies**.
- **Refactor the power-up supply creation logic**, ensuring all power-up instances are created through factories.

Experiment Steps: Factory Pattern (6)

➤ How to use Factory Pattern to create enemy aircraft and power-ups?



Experiment Steps: Factory Pattern Example (1)

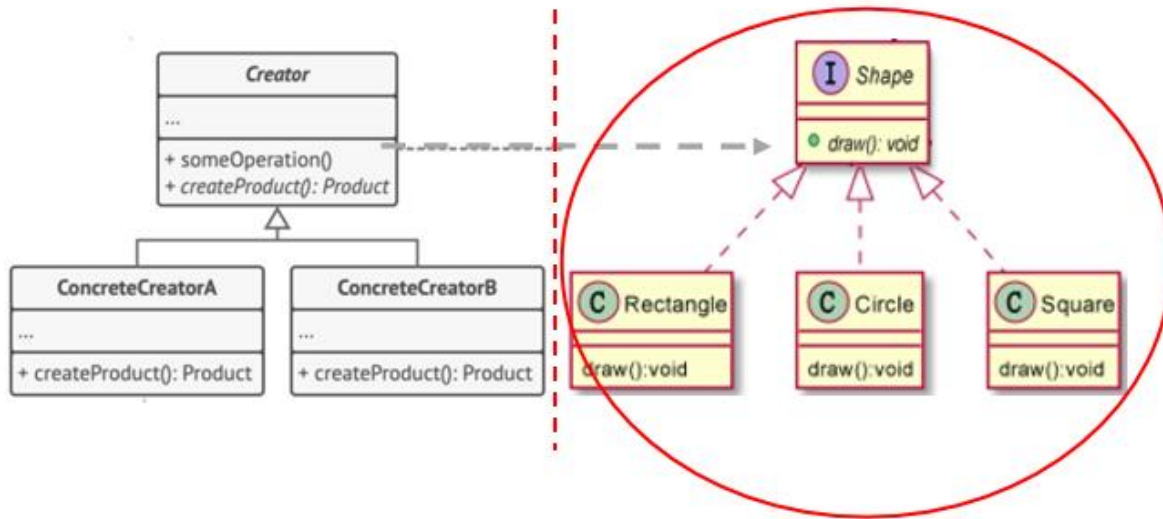
Suppose we want to develop a **graphics factory** that can produce **three types of graphic products**: circles, rectangles, and squares.

How should we implement this using the Factory Pattern?



Experiment Steps: Factory Pattern Example (2)

Product Line: Create **Shape** interface and 3 concrete shape classes that implement it



① Create an interface to act as the Product role

```
public interface Shape {
    void draw();
}
```

② Create entity classes that implement the interface, acting as Concrete Product roles

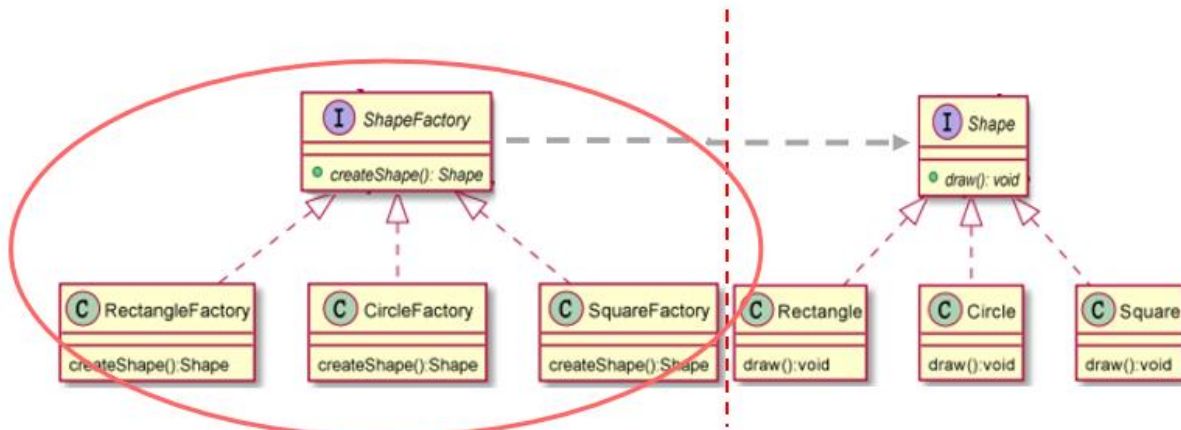
```
public class Rectangle implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Rectangle::draw() method.");
    }
}
```

```
public class Square implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Square::draw() method.");
    }
}
```

```
public class Circle implements Shape {
    @Override
    public void draw() {
        System.out.println("Inside Circle::draw() method.");
    }
}
```

Experiment Steps: Factory Pattern Example (3)

Factory Line: Create **ShapeFactory** interface and 3 concrete factories implementing it



③ Create factory interface to act as the Creator role

```
public interface ShapeFactory {  
    public abstract Shape createShape();  
}
```

④ Create concrete factories implementing the factory interface, acting as Concrete Creators

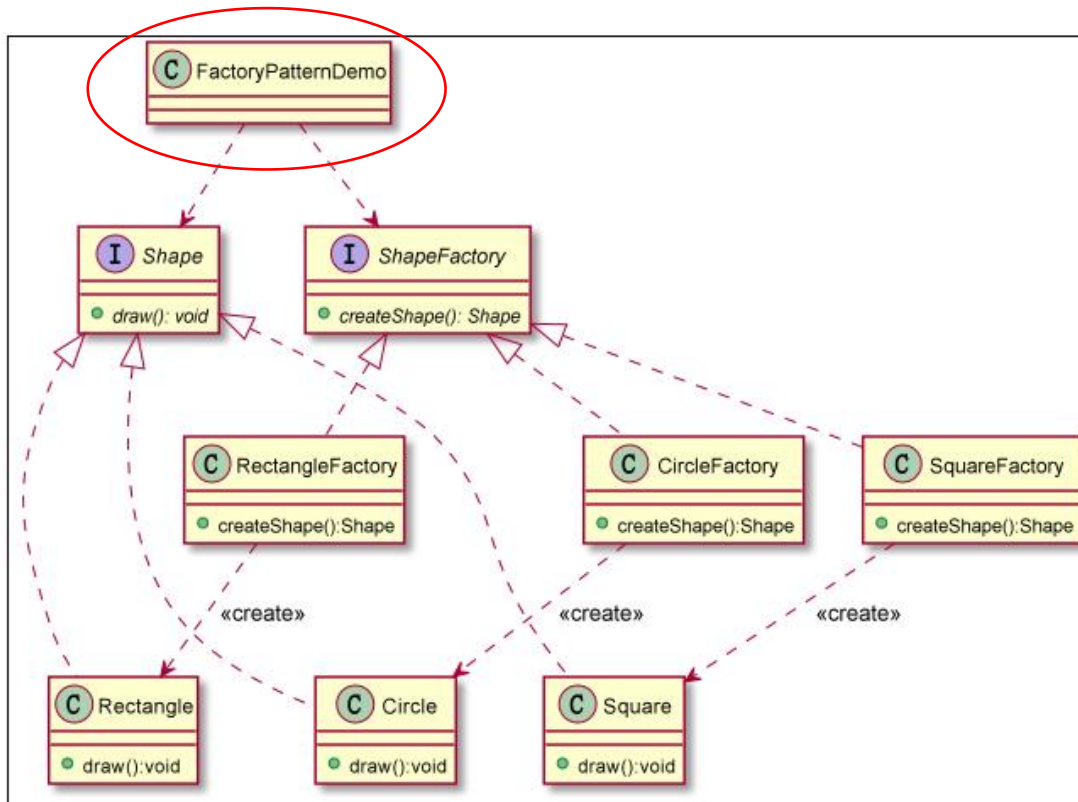
```
public class RectangleFactory implements ShapeFactory {  
    @Override  
    public Shape createShape() {  
        return new Rectangle();  
    }  
}
```

```
public class SquareFactory implements ShapeFactory {  
    @Override  
    public Shape createShape() {  
        return new Square();  
    }  
}
```

```
public class CircleFactory implements ShapeFactory {  
    @Override  
    public Shape createShape() {  
        return new Circle();  
    }  
}
```

Experiment Steps: Factory Pattern Example (4)

⑤ Use FactoryPatternDemo to demonstrate the usage of the Factory Pattern



```
public static void main(String[] args) {
```

```
    ShapeFactory shapeFactory ;
    Shape shape;
```

```
    //获取 Circle 的对象，并调用它的 draw 方法
    shapeFactory = new CircleFactory();
    shape = shapeFactory.createShape();
    shape.draw();
```

```
    //获取 Rectangle 的对象，并调用它的 draw 方法
    shapeFactory = new RectangleFactory();
    shape = shapeFactory.createShape();
    shape.draw();
```

```
    //获取 Square 的对象，并调用它的 draw 方法
    shapeFactory = new SquareFactory();
    shape = shapeFactory.createShape();
    shape.draw();
```

```
}
```

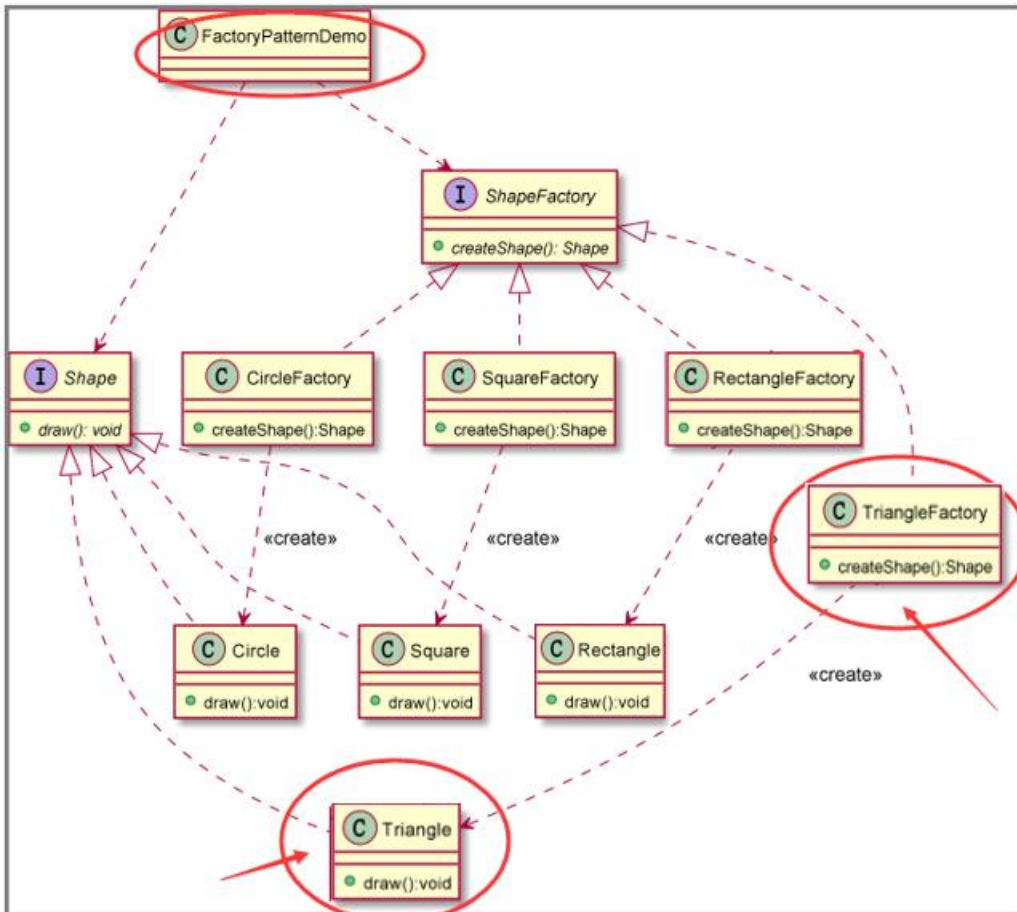
```
Inside Circle::draw() method.
Inside Rectangle::draw() method.
Inside Square::draw() method.
```

Polymorphism: the same interface, different behaviors.

Experiment Steps: Factory Pattern Example (5)



Let's think: How to add a triangle?



```
public class FactoryPatternDemo {

    public static void main(String[] args) {

        ShapeFactory shapeFactory ;
        Shape shape;

        //Obtain the Circle object and call its draw method
        shapeFactory = new CircleFactory();
        shape = shapeFactory.createShape();
        shape.draw();

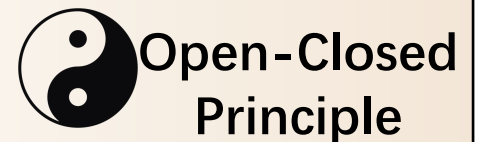
        //Obtain the Rectangle object and call its draw method
        shapeFactory = new RectangleFactory();
        shape = shapeFactory.createShape();
        shape.draw();

        //Obtain the Square object and call its draw method
        shapeFactory = new SquareFactory();
        shape = shapeFactory.createShape();
        shape.draw();

        //Obtain the Triangle object and call its draw method
        shapeFactory = new TriangleFactory();
        shape = shapeFactory.createShape();
        shape.draw();

    }

}
```



Assignment Submission

- **Submission requirements**

- ① Project code (the entire project folder)
- ② Experiment report (use report template)

- **Deadline**

Submit your work to the **HITsz Grader** assignment submission platform by the deadline: Tuesday of Week 14.

Login URL: <https://grader.cs-lab.top/>

The default username and password are your student ID.



Let's begin the experiment!

THANK YOU